

LQCD_Master 核心部分穷尽剖析报告

双代理编排 × 物理任务 × Wick 收缩引擎的代码-物理交叉向深挖

opencode pure

2026 年 8 月 17 日

摘要

本报告对 `refer/git-rep/LQCD_Master` 做只读穷尽剖析。性质判定：代码-物理交叉向——agent 编排代码（约 26 万行核心）以 LQCD 物理任务（benchmark 15144 行 + skills 物理推导 3348 行）为输入与验证基准，核心机制是”物理任务 → 物理方案 YAML → 物理代码 `main.py`”的逐符号映射。穷尽枚举 **13 个核心部分候选**（C1–C13），三态分配：深挖 **6** / 浅析 **4** / 排除 **3**。最深剖析结论：`generate_einsum Wick 收缩引擎`（自动 Wick 配对 + γ 代数 + 费米子符号，实测正确输出 $\pi/\rho/\eta_s$ （含 disconnected）/ p/Ω_c 的 einsum）与 `双代理 solve/critique/rewrite 循环 + 技能路由注入` 是两大独特竞争力，支撑 70 任务基准中标准任务近满分（90.0%）的实测表现。

目录

1	项目定位与核心部分清单	1
1.1	项目性质判定	1
1.2	核心部分总清单（穷尽枚举）	1
2	核心部分深度剖析	2
2.1	C1 双代理分工与 prompt 工程	2
2.2	C2 5 个 LQCD 技能设计	3
2.3	C3 benchmark 物理任务与技能调用链	5
2.4	C4 core_architecture 架构设计	5
2.5	C5 run.py 入口与配置体系	6
2.6	C6 generate_einsum Wick 收缩代码生成引擎	7
3	独特性与竞争力评估	8
4	关键片段与公式	9
5	参考源清单	14
6	结论	16

1 项目定位与核心部分清单

1.1 项目性质判定

要点

项目性质: 代码-物理交叉向。依据 (实测): ① 代码体量编排/工具/提示词/配置代码约 26 万行 (run.py + core_architecture 2415 + prompts 527 + utils 260521 + configs 212); ② 物理内容 benchmark 物理任务与参考实现 15144 行、skills 物理推导 3348 行; ③ 映射机制双代理把物理任务转译为物理方案再转译为物理代码, skills 编码 Wick 收缩/谱分解推导, generate_einsum 把物理算符自动映射为 einsum 字符串。代码是载体、物理是输入与标尺, 二者靠一一对应的映射链耦合——故判交叉向。

1.2 核心部分总清单 (穷尽枚举)

编号	核心部分	处理态	独特性概述
C1	planner+executor 双代理 prompt 工程	深挖	solve/critique/rewrite 循环 + 物理/代码 双层评审
C2	5 个 LQCD 技能设计	深挖	SKILL.md 编码物理推导链 + 代码-物理 映射
C3	benchmark 物理任务与技能调用链	深挖	70 任务自然语言基准 + 手写参考数值
C4	core_architecture 架构设计	深挖	编排器三阶段 + checkpoint 人机协同
C5	run.py 入口与配置体系	深挖	极薄入口 + 配置驱动提交渲染
C6	generate_einsum Wick 收缩引擎	深挖	自动 Wick 配对/ γ 代数/费米子符号的代 码生成器
C7	static_check 静态分析	浅析	AST 级 PyQUDA 语义检查 (辅助纠错)
C8	SkillRouter/SkillRunner 路由注入	浅析	技能按阶段注入 LLM (C2 的机制层)
C9	experiments 多骨干实验体系	浅析	GPT-5.4/DeepSeek 对比验证闭环
C10	llm_client / tool_client	浅析	OpenAI 兼容客户端与 web 搜索/执行工 具
C11	10 万行级自动生成 sink 大文件	排除	自动生成产物 (C6 输出), 非设计部分
C12	docs 既有 pure 报告	排除	第三方分析产物, 非库功能
C13	.env / requirements 脚手架	排除	通用样板, 无独特竞争力

表 1: 核心部分总清单 (三态填满, 无”未处理”; 数据来源: 实
测索引)

2 核心部分深度剖析

六个”深挖”候选按 15 步框架逐部分重现（交叉向：映射表 + 逐符号解释为主，物理元素并入对应步骤）。

2.1 C1 双代理分工与 prompt 工程

- 1. 目的与前期准备.** 双代理把”物理正确性”与”代码正确性”分离验证：Planner 只产物理方案、Executor 只产代码，避免单模型同时承担两难任务（README 定位：Planner 产方案、Executor 产代码，见 README.md:12）。[确证]
- 2. 输入.** Planner 吃自然语言任务 + 固定配置事实 + 技能（core_architecture/planner.py:72-81）；Executor 吃 plan_yaml + summary_md + 输入事实 + 提交配置（core_architecture/executor.py:102-159）。[确证]
- 3. 输出.** Planner 输出 plan.yaml + summary.md；Executor 输出含 main_program、test_submit、full_submit、notes 四字段的 JSON（输出格式强制 prompts/executor/generate_system.md:140-160）。[确证]
- 4. 整体框架.** 每代理均”生成 → 批判 → 改写”三子阶段：Planner 为 solve/critique/rewrite（core_architecture/planner.py:72-112），Executor 为 generate/critique/static-check/rewrite（core_architecture/executor.py:398-524）。[确证]
- 5. 关键细节.** ① 批判只标”会导致代码失败或物理错误”的致命问题（prompts/planner/critique_system.md:6-9）；② Executor 提示词强制单文件顺序脚本、禁止 main 函数/类/异常处理（prompts/executor/generate_system.md:34-46）；③ generate_einsum 工具强制调用 + 水印检查（prompts/executor/generate_system.md:63-82）。[确证]
- 6. 正确执行.** 每代理生成后经编排器 checkpoint（人工确认/反馈），未通过则 rewrite_with_feedback 进入下一版本，上限 max_revision_rounds=3（core_architecture/orchestrator.py:109-133）。[确证]
- 7. 实际执行.** 只读侧实测：69 个核心 py 语法检查 OK=69/FAIL=0；双代理真实 LLM 调用 **未验证**（需 API key）。[确证/未验证]
- 8. 实际输出.** 仓库内真实实验目录含 executor_v1 完整产物（experiments/LQCD_Master_GPT_5.4/2pt_local/task_1/executor_v1/）。[确证]
- 9. 结果分析.** 70 任务 63 exact / 3 sign-flip / 4 fail (90.0%)，2pt local 20/20 满分；3pt baryon 仅 73.3%——重子 3pt 顺序源拓扑是 prompt 工程仍未完全收敛的难点（experiments/LQCD_Master_GPT_5.4/summary.md:8-30）。[确证]
- 10. 综合分析.** 对象映射：自然语言 ↔ 物理任务；critique ↔ 同行评审；rewrite ↔ 论文修改；checkpoint ↔ 人机协同护栏。深层原因：把 LQCD 领域协作惯例（审稿 + 修改 + 确认）编码为软件流程。[确证/推断]
- 11. 总结.** 双代理 + 三子阶段 + 人工 checkpoint 构成”自动评审、人机兜底”的工程闭环，是 90% 级准确率的结构性的保证。[确证]
- 12. 评价.** 优：物理/代码解耦、可复现；缺：批判偏保守（只拦致命错误）、无符号约定预防机制。[推断]

13. **补充.** prompt 以英文编写、面向 GPT/DeepSeek 类通用 LLM，未针对推理链/长上下文做显式优化（推断）。[推断]

14. **参考源.** 见第 5 节 C1 条目。

15. **附录.** prompt 原文片段见第 4 节 S3/S4。

2.2 C2 5 个 LQCD 技能设计

1. **目的与前期准备.** 把领域知识（物理推导、PyQUDA 用法、分析流程）编码为可注入 LLM 的独立模块，替代”硬编码在 prompt 里”的可维护性差的方案。前置：技能 = 含 YAML front-matter 的 markdown (utils/skill_utils.py:93-111 解析)。[确证]

2. **输入.** 5 个 SKILL.md 共 1926 行：lqcd-analysis(740)/pyquda-tool(614)/ pyquda-gauge(222)/lqcd-physics-correlator(199)/lqcd-physics-spectrum(151) (skills/lqcd-analysis/SKILL.md:1、skills/pyquda-tool/SKILL.md:1)。[确证]

3. **输出.** 注入后的 system prompt 块（含 Description + body + 内联 reference，上限 24000 字符，utils/skill_utils.py:215-247）。[确证]

4. **整体框架.** 分工链：correlator（算符/关联函数）→ spectrum（谱分解/拟合模板）→ analysis（数据分析）；工具侧 pyquda-tool（传播子/收缩）/ pyquda-gauge（纯规范观测量）。物理链尾部显式移交下家（skills/lqcd-physics-correlator/SKILL.md:36-41）。[确证]

5. **关键细节.** correlator 技能给出 Wick 收缩关键公式与 γ_5 -厄米约定（ $\text{prop} = \gamma_5 S^\dagger \gamma_5$ ）与 flavor 对称性约化（skills/lqcd-physics-correlator/SKILL.md:88-98）；DeGrand-Rossi 基 γ 矩阵完整列出（skills/lqcd-physics-correlator/SKILL.md:52-63）。[确证]

代码-物理映射

映射原则: 每个技能中的物理符号都能落到生成的代码符号，双向可查。

代码符号	物理对象	物理含义与参考源
prop_l/prop_s/prop_c	轻/奇异/粲夸克传播子	$S_l(x, y)$ 等; skills/lqcd-physics-correlator/SKILL.md:23
G5	γ_5	赝标量算符与 γ_5 -厄米约定; skills/lqcd-physics-correlator/SKILL.md:60
epsilon	ϵ^{abc} 颜色反对称张量	重子 diquark 色结构; utils/generate_einsum/hadron_operator.py:20-23
Tmat	投影子	正宇称重子投影 $(1 + \gamma_t)/2$; benchmark/3pt_baryon/task/task_1.txt:3
covDev	协变位移算子	非局域算符的 Wilson 线位移; prompts/executor/generate_system.md:119

表 2: 技能层代码-物理映射（数据来源: 实测）

6. **正确执行.** 技能经 SkillMessageAssembler 拼入 system 消息, reference 目录内容内联 (utils/skill_

utils.py:249-262); 路由由 skills.yaml 控制 (configs/skills.yaml:1-10)。[确证]

7. 实际执行. 只读侧逐技能精读 (correlator 全读 + 其余头部); 对 5 个 SKILL.md 统计 front-matter/正文结构 (实测)。真实 LLM 注入效果 **未验证**。[确证/未验证]

8. 实际输出. 每个技能带 reference/ 工作示例 (如 pion/proton mass、wilson loop、baryon 3pt 代码), 是”技能 → 代码”的示范性输出 (skills/lqcd-physics-correlator/SKILL.md:132-134)。[确证]

9. 结果分析. 技能路由强制 planner 吃物理技能、executor 吃工具技能 (configs/skills.yaml:4-8), 与实测准确率对照: 纯规范任务 100%、重子 3pt 最低——技能覆盖度与任务难度匹配。[推断]

10. 综合分析. 物理公式 (如 $C_2(t) = \langle O(t)O^\dagger(0) \rangle$, (1)) 以 LaTeX 写入技能, LLM 据此推理; 代码符号映射表 (上文) 保证”物理 ↔ 代码”逐项可查。[确证]

$$C_2(\vec{p}; t_f, t_i) = \langle \mathcal{O}(\vec{p}, t_f) \mathcal{O}^\dagger(\vec{p}, t_i) \rangle \quad (1)$$

11. 总结. 技能库是”物理知识 → 可注入模块”的编译结果, 5 技能构成从观测量到代码的完整推理链。[确证]

12. 评价. 优: 物理推导以标准 LaTeX 呈现、带 reference 示例; 缺: 技能间依赖靠文本移交, 无机制性校验。[推断]

13. 补充. 技能内容本身即”代码-物理交叉”最密集处, 是本库物理承载主体。[确证]

14. 参考源. 见第 5 节 C2 条目。

15. 附录. SKILL.md 关键片段见第 4 节 S6/S7。

2.3 C3 benchmark 物理任务与技能调用链

1. 目的与前期准备. 以”手写标准 PyQUDA 参考实现 + 数值”为 gold standard, 量化 agent 生成代码的正确性 (README.md:118)。前置: C24P29 ensemble、cfg 10000、点源、零动量统一评测条件 (README.md:108)。[确证]

2. 输入. 70 个自然语言任务文本 (2pt_local 20 / 2pt_nonlocal 10 / wilson_loop 12 / 3pt_meson 13 / 3pt_baryon 15, README.md:108-118)。[确证]

3. 输出. standard/benchmark.py (手写参考) + benchmark_data/*.txt (参考数值) 供逐任务比对 (benchmark/2pt_local/standard/benchmark.py:1-30)。[确证]

4. 整体框架. 调用链: task.txt → run.py → Planner → Executor (+skills+generate_einsum) → main.py 运行 → txt 数据 → 与 benchmark_data 对比。[确证]

5. 关键细节. 任务文本既含物理量 ($\bar{u}\gamma_5 d$ 、Cg5、Tmat) 又含数值参数 (stout (1,0.125,4)、tseq=8、点源 [0,0,0,0]), 是”物理-工程”双层指令 (benchmark/3pt_baryon/task/task_1.txt:1-8)。[确证]

6. 正确执行. 每类任务一个 batch 脚本批量跑 (如 experiments/run_scripts/batch_local2pt.sh: 1)。[确证]

7. 实际执行. 只读侧抽样精读 2pt_local/wilson_loop/3pt_meson/3pt_baryon 四类 task_1 + standard 参考头部; 全量 70 任务仅索引。[确证]

8. 实际输出. standard 参考数据含逐任务 t=1 Re 数值 (如 DeepSeek 失败表中 $\eta_s + 1.3067e-01$ 等, experiments/LQCD_Master_DeepSeek/summary.md:20-28)。[确证]

9. 结果分析. Exact 判据 $|\Delta| < 10^{-12}$ 或 $\text{rel} < 10^{-3}$; sign-flip 单独计数 (README.md:131-133)。wilson_loop 全满分说明纯规范任务技能 (pyquda-gauge) 覆盖充分。[确证/推断]

10. **综合分析.** 基准既是验证集也是”物理正确性定义”——agent 的”物理”由手写标准编码, 代码-物理对应在此闭环 (benchmark/2pt_local/standard/benchmark.py:1-18)。[确证]
11. **总结.** 70 任务基准把”agent 算得对不对”从印象化转为数值可判, 是系统可信度的支柱。[确证]
12. **评价.** 优: 任务覆盖 5 类 observable、标准全手写; 缺: 单 ensemble 单组态 (cfg 10000), 统计意义有限 (推断)。[推断]
13. **补充.** 任务文本仅 1-10 行, 恰好是”物理-工程双层”最小完备指令集。[确证]
14. **参考源.** 见第 5 节 C3 条目。
15. **附录.** task_1 原文见第 4 节 S8。

2.4 C4 core_architecture 架构设计

1. **目的与前期准备.** 三层 (orchestrator/planner/executor) 封装”总控/方案/代码”三个关注点 (行数 816/379/1219, 职责行数比直观反映”方案最薄、代码最厚”)。[确证]
2. **输入.** task + run_dir + llm_client + tool_client + submit_tool + 模式标志 (core_architecture/orchestrator.py:30-49)。[确证]
3. **输出.** state dict (planner/executor/test_submission/full_submission 各节) 与 trajectory_full.json (core_architecture/orchestrator.py:62-96)。[确证]
4. **整体框架.** run() 三阶段: planner → executor → test submission (core_architecture/orchestrator.py:74-96)。[确证]
5. **关键细节.** ① 每阶段 checkpoint 循环 (approved/feedback) 上限 3 轮 (core_architecture/orchestrator.py:109-121); ② test 提交后 squeue/scontrol 监视 + 动态重写 (core_architecture/orchestrator.py:201-281、core_architecture/orchestrator.py:283-343); ③ executor 缺字段自动反馈重试上限 2 (core_architecture/executor.py:165-197)。[确证]
6. **正确执行.** orchestrator 自身不产生 prompt 内容, 全部委托 planner/executor 与技能/工具层, 只做状态机与 I/O (core_architecture/orchestrator.py:98-107)。[确证]
7. **实际执行.** 只读侧全函数结构 grep + 关键段精读; 运行态执行 **未验证**。[确证/未验证]
8. **实际输出.** 实验产物中 planner/ 与 executor_v1..v2 目录结构即编排器”版本化 + 持久化”设计的落地 (experiments/New_DA_diagonal/planner/plan.yaml:1)。[确证]
9. **结果分析.** executor 静态分析轮数/动态测试轮数由 config 控制 (executor_static_check_rounds=1、executor_dynamic_test_rounds=1, configs/config.yaml:1-2), 架构预留了自动纠错深度的可调性。[推断]
10. **综合分析.** 状态机设计使每一步产物 (plan/bundle/queue/monitor) 皆落盘, 为实验层 5335 文件的可追溯性提供机制保障 (core_architecture/orchestrator.py:815)。[确证/推断]
11. **总结.** 编排器是”薄总控 + 厚实现”, 把可靠性建立在持久化与版本化上。[确证]
12. **评价.** 优: 状态可恢复、监视细节丰富; 缺: orchestrator 816 行承载过多 Slurm 解析逻辑, 可抽独立模块 (推断)。[推断]
13. **补充.** 设计面向单任务串行; 未做任务级并行编排 (推断)。[推断]
14. **参考源.** 见第 5 节 C4 条目。
15. **附录.** orchestrator.run 片段见第 4 节 S1。

2.5 C5 run.py 入口与配置体系

- 1. 目的与前期准备.** 极薄入口 (133 行) 把”装配 + 启动”与”实现”分离; 配置体系让集群适配不侵入代码。前置: 配置 yaml 需逐集群定制 (README.md:150-157)。[确证]
- 2. 输入.** CLI 五参数 (-test/-run-dir/-task/-list-skills/-non-interactive, run.py:17-25) + .env (API key) + configs 三 yaml。[确证]
- 3. 输出.** run_dir (默认 runs/< 时间戳 >) 下的全部产物 (run.py:28-32)。[确证]
- 4. 整体框架.** main() 装配四组件后调 orchestrator.run(), 结束打印轨迹路径与版本/提交摘要 (run.py:90-129)。[确证]
- 5. 关键细节.** ① -task 支持文件 (.txt 后缀自动读入, run.py:95-98); ② 交互输入用 prompt_toolkit、回退 input (run.py:61-71); ③ config.yaml 的 slurm/runtime/ensemble 节直接驱动提交脚本渲染 (core_architecture/executor.py:1113-1160)。[确证]
- 6. 正确执行.** 快速冒烟: python run.py -list-skills (只列技能, 不发 LLM 请求); 全流程: python run.py -task "< 文本 >" -test -non-interactive (README.md:26-28)。[确证]
- 7. 实际执行.** 只读侧实测: CLI 结构精读、config 三文件全文/结构精读; 实际运行 (需 API+ 集群) 未验证。[确证/未验证]
- 8. 实际输出.** run.py 打印的 JSON 摘要字段 (planner/executor 版本、test/full submission) 与实际实验目录结构一致 (run.py:124-129)。[确证]
- 9. 结果分析.** ensemble_presets.yaml 提供 7 套 ensemble 预设, benchmark 与实验用 C24P29 参数即取自该体系 (benchmark/2pt_local/standard/benchmark.py:17-25)。[确证/推断]
- 10. 综合分析.** 入口薄 + 配置厚的设计使”换集群只改 yaml 不改代码”, 是面向多机构部署的务实取舍。[推断]
- 11. 总结.** 入口与配置是系统的”外围适配层”, 决定部署成本。[确证]
- 12. 评价.** 优: 零配置即可 -list-skills 冒烟; 缺: config 无 schema 校验, 拼写错误要到渲染时才暴露 (推断)。[推断]
- 13. 补充.** .env 仅 LLM/Serper key, 无集群凭据——提交依赖 Slurm 环境预配置 (.env.template:1-8)。[确证]
- 14. 参考源.** 见第 5 节 C5 条目。
- 15. 附录.** config.yaml 片段见第 4 节 S9。

2.6 C6 generate_einsum Wick 收缩代码生成引擎

核心算法

核心算法: 从物理算符 (meson/baryon/multi-hadron) 出发, 经 wicklib Correlator 自动 Wick 配对 + γ 代数化简 + 费米子符号计算, 生成 PyQUDA 风格 contract() einsum 字符串; 实测 $\pi/\rho/\eta_s$ (含 disconnected)/ p/Ω_c 正确输出。

- 1. 目的与前期准备.** 手写 Wick 收缩极易出错 (符号、色/旋指标、 γ 顺序), 该引擎把”算符 $\rightarrow$ einsum”自动化, 并作为 executor 的强制工具 (prompt 层强制调用 + 水印校验, prompts/executor/generate_system.md:63-82)。[确证]

2. **输入.** 算符描述: meson (flavor+ γ)、baryon (三个 flavor+ 投影子 +diquark γ)、multi-hadron (specs 列表)、3pt (源/汇/流 + tseq) (utils/generate_einsum/hadron_operator.py:1-40)。[确证]
3. **输出.** 含 # FROM generate_einsum 水印的代码块 + sink 文件 (multi-hadron 场景产出独立 sink_XXX.py) (prompts/executor/generate_system.md:52-60)。[确证]
4. **整体框架.** 三段: 算符建模 (Tensor 列表) $\rightarrow$ Wick 配对引擎 (wicklib Correlator, utils/generate_einsum/wicklib/correlator.py:1-30) $\rightarrow$ einsum 格式化 (codegen, utils/generate_einsum/two_pt/codegen.py:1-30)。[确证]
5. **关键细节.** ① γ 变量名约定集 `_GAMMAS=G5,g1..g4,gtg5,Cg5,I4,Cg1` (utils/generate_einsum/two_pt/codegen.py:16-18); ② 反向传播子自动 `.dag()` $\rightarrow$ `.conj()` 转换 (utils/generate_einsum/two_pt/codegen.py:47-54); ③ disconnected 图自动标记 (η_s 孤环, 实测输出)。[确证]
6. **正确执行.** 2pt 用 `wick_contract_2pt`; 3pt 用 `meson_3pt/baryon_3pt` 生成器 (含 sink block + 顺序源 + 终收缩, utils/generate_einsum/three_pt/codegen_meson.py:1-30)。[确证]
7. **实际执行.** 只读实测: 运行 `demo_2pt.py` (exit=0, 输出不落盘, `__pycache__` 已清理)。[确证]
8. **实际输出.** 实测生成:

```

1 # pi pi = ubar . g5 . d
2 contract('wtzyxCBba, wtzyxCBba -> t', prop_l.data.conj(), prop_l.data)
3 # eta_s (2 terms, 含 disconnected)
4 [DISCONNECTED] -contract('wtzyxBAaa, wtzyxBAaa -> t', prop_s.data, prop_s.data)
5 contract('wtzyxCBba, wtzyxCBba -> t', prop_s.data.conj(), prop_s.data)
6 # proton (2 terms, epsilon+Cg5+Tmat)
7 -contract('AB, abc, EF, efd, CD, wtzyxDBdb, wtzyxFafa, wtzyxECec -> t',
8           Cg5, epsilon, Cg5, epsilon, Tmat, prop_l.data, prop_l.data, prop_l.data)

```

(数据来源: 本会话实测, 命令与完整输出见会话日志) [确证]

9. **结果分析.** 与手写标准对照: π 的 `contract('wtzyxCBba, wtzyxCBba -> t', ...)` 与 benchmark 参考的 $\text{Tr}[S^\dagger \cdot S]$ 形式一致 (benchmark/2pt_local/standard/benchmark.py:1-18); η_s 自动识别 disconnected 项说明 Wick 配对含孤环处理。[确证]
10. **综合分析.** einsum 表达式 (2) 即”传播子、 γ 矩阵、指标求和约定”的压缩编码; 引擎把物理推导 (Wick 定理) 落为可执行代码, 是”代码-物理交叉”最纯的实现。[确证]

$$\text{contract}('wtzyxCBba, wtzyxCBba \rightarrow t', S^\dagger, S) \iff C_\pi(t) = \sum_{\vec{x}} \text{Tr}[S_l^\dagger(\vec{x}, t) S_l(\vec{x}, t)] \quad (2)$$

11. **总结.** generate_einsum 是”物理算符 $\rightarrow$ 正确收缩代码”的决定性依赖, 实测正确性支撑 executor 的 2pt/3pt 高准确率。[确证]
12. **评价.** 优: 覆盖 meson/baryon/multi-hadron/3pt 全谱; 生成文件可达 10 万行级 (多强子 9 夸克收缩) 说明拓扑规模已远超手写能力。缺: 生成 sink 文件体积失控、旧版 `_to_delete` 归档堆积 (推断)。[确证/推断]
13. **补充.** 引擎依赖 numpy/opt_einsum 与 wicklib 内部代数; 独立运行模式经 sys.path 注入 (utils/generate_einsum/two_pt/codegen.py:3-11)。[确证]
14. **参考源.** 见第 5 节 C6 条目。
15. **附录.** demo_2pt.py 驱动主流程片段见第 4 节 S10; wicklib 引擎片段见 S11。

3 独特性与竞争力评估

独特点	对比基准	优势与证据
Wick 收缩自动生成引擎	手写收缩/通用符号库	算符 $\rightarrow$ einsum 自动化，实测 5 类粒子正确；水印强制防手写 (prompts/executor/generate_system.md:63-82)
双代理 + 三子阶段循环	单次生成 agent	物理/代码双层评审 + 自动重写；70 任务 90.0% (experiments/LQCD_Master_GPT_5.4/summary.md:8-30)
技能库注入路由	固定 prompt	5 技能按阶段注入、reference 内联；物理推导 LaTeX 化 (utils/skill_utils.py:215-247)
PyQUDA 语义静态检查	纯语法检查	AST 级 API 语义分析，错误进 critique 自动修复 (utils/static_check.py:186-265)
70 任务手写数值基准	无基准 agent	数值可比对的验证闭环 (README.md:108-118)
人机协同 checkpoint	全自动 agent	提交前强制人工确认，安全兜底 (README.md:57-61)

表 3: 独特性评估（数据来源: 实测/推导）

4 关键片段与公式

```
1 def run(self) -> dict[str, Any]:
2     print("===== [ Planner ] =====")
3     planner_output, planner_version = self._run_planner_stage()
4     print()
5     print("===== [ Executor ] =====")
6     bundle, exec_version = self._run_executor_stage(planner_output)
7     if self.test_mode:
8         self.state["test_submission"] = {
9             "approved": False,
10            "result": {"skipped": True, "reason": "test_mode"},
11        }
12        self.state["full_submission"] = {
13            "approved": False,
14            "result": {"skipped": True, "reason": "not_submitted_in_main_workflow"},
15        }
16        self._save_state()
```

```

17         else:
18             self._run_test_submission_stage(bundle, exec_version, planner_output)
19
20         self.state["finished_at"] = datetime.now().isoformat()
21         self.state["status"] = "completed"
22         self._save_state()
23         return self.state

```

```

1     def run(self, task: str) -> dict[str, Any]:
2         self._begin_iteration(version_label="v1", kind="initial", feedback=None)
3         self.solve_payload = self._run_stage(
4             stage="solve",
5             build_messages=lambda: self._build_stage_user_message(
6                 stage="planner_solve",
7                 payload={"task": task, "fixed_facts_yaml": self.fixed_facts_yaml},
8                 task=task,
9             ),
10            task=task,
11        )
12
13        self.critique_payload = self._run_stage(
14            stage="critique",
15            build_messages=lambda: self._build_stage_user_message(
16                stage="planner_critique",
17                payload={"task": task, "solve_payload": self.solve_payload},
18                task=task,
19            ),
20            task=task,
21        )
22        self.critique_payload = self._normalize_critique_payload(self.critique_payload)
23
24        self.rewrite_payload = self._run_stage(
25            stage="rewrite",
26            build_messages=lambda: self._build_stage_user_message(
27                stage="planner_rewrite",
28                payload={
29                    "task": task,
30                    "solve_payload": self.solve_payload,
31                    "critique_payload": self.critique_payload,
32                    "fixed_facts_yaml": self.fixed_facts_yaml,
33                },
34                task=task,
35            ),
36            task=task,
37        )
38
39        output = self._to_planner_output(self.rewrite_payload, fallback=self.solve_payload)
40        self._persist_final(output)

```

- 1 You are a senior lattice QCD (LQCD) expert in high-energy physics with:
- 2 1. a strong theoretical physics background: familiar with QCD, Euclidean lattice path integrals, hadron spectroscopy,
 ↪ matrix elements, thermal QCD, topology, and numerical gauge-field computations;
 - 3 2. a strong numerical computing background: familiar with the full workflow of ensembles, gauge configurations, Dirac
 ↪ solvers, source-sink construction, contractions, jackknife/bootstrap, excited-state contamination, and
 ↪ renormalization;

```

4 3. strong engineering execution ability: able to transcribe physics tasks into a clear, executable, and extensible
   ↳ YAML plan to guide collaborators in implementing PyQUDA programs and submitting them to a cluster.
5 4. plan structure awareness: able to distinguish standard LQCD tasks (hadron 2pt/3pt/nonlocal 2pt, requiring the
   ↳ hadrons→propagators→correlators chain) from non-standard tasks (topological charge, Polyakov loop, static
   ↳ potential, etc.).
6 - For standard tasks, use task_mode: standard and populate the normal physics sections.
7 - For non-standard tasks, use task_mode: freeform and write the full physics description in freeform_plan, leaving
   ↳ the standard sections minimal or empty.
8 Using standard sections for non-standard tasks produces misleading plans and harms code generation.
9
10 Your core mission is to apply reasoning paradigm of a physicist:
11 - identify which category of physics problem the task belongs to and apply relevant domain knowledge deeply;
12 - make reasonable detail plans, designing the physical quantity, operator, source-sink design, lattice parameters,
   ↳ statistical strategy, and systematic uncertainty control;
13 - finally output a concise plan that is physically reasonable, numerically executable, and engineering-ready.

```

```

1 Code structure and writing order:
2
3
4 - the script should be a single-file sequential execution, without a `main` function;
5 - do not define any helper functions;
6 - write in the following order (this is the universal skeleton for ALL LQCD code):
7   1. parameter definitions (hard-coded)
8   2. read gauge configuration
9   3. construct the Dirac operator (skip if gauge-only)
10  4. compute forward propagators (skip if gauge-only)
11  5. extract observable / compute contraction (task-specific)
12  6. save the result

```

```

1 ## Core workflow for Correlators
2
3 ### Step 1: Identify the interpolating operator(s)
4
5 For a target hadron with quantum numbers  $J^{PC}$  and flavor content, write the interpolating operator. Use Dirac
   ↳ bilinears for mesons, and appropriate diquark-quark structures for baryons.
6
7 For example, the simplest interpolating operator for a  $\pi^+$  meson is usually written as  $\mathcal{O}_{\pi^+} = \bar{d} \gamma_5 u$ 
   ↳  $\{d\}^a \gamma_5 u^a$ , and the simplest operator for a proton is  $\mathcal{O}_p = \epsilon^{abc} (u^a C \gamma_5 d^b) u^c$ . The simplest local operators are usually sufficient for ground state mass extraction, but
   ↳ interpolating operators can be constructed with gamma matrices and gauge covariant derivatives to access
   ↳ different quantum numbers, excited states, and observables related to hadron structure in general. For
   ↳ example, if we want to compute pion quasi-distribution amplitudes, we should use non-local operators with
   ↳ quark fields separated by a Wilson line, e.g.  $\mathcal{O}_{\pi^+}(x;z) = \bar{d}^a(0) \gamma_5 W(0,z) u^a(z)$ 
   ↳  $\cdot$ . If we want to compute a matrix element of an axial vector current inserted on the  $u$  quark, we should
   ↳ insert the current operator  $J_\mu = \bar{u} \gamma_\mu \gamma_5 u$  between the source and sink operators in the
   ↳ three-point function.
8
9 **Gamma matrices convention**: Use the DeGrand-Rossi basis as the Euclidean Dirac basis, which is the default gamma
   ↳ basis in PyQUDA convention, can be defined in terms of the Pauli matrices  $\sigma_i$  as:
10
11
12 $$
13 \gamma_1 = \begin{pmatrix} 0 & i \sigma_1 \\ -i \sigma_1 & 0 \end{pmatrix}, \text{quad}
14 \gamma_2 = \begin{pmatrix} 0 & -i \sigma_2 \\ i \sigma_2 & 0 \end{pmatrix}, \text{quad}
15 \gamma_3 = \begin{pmatrix} 0 & i \sigma_3 \\ -i \sigma_3 & 0 \end{pmatrix}, \text{quad}

```

```

16 \gamma_4 = \begin{pmatrix} 0 & I_{\{2 \times 2\}} \\ I_{\{2 \times 2\}} & 0 \end{pmatrix}, \quad \\
17 \gamma_5 = \gamma_1 \gamma_2 \gamma_3 \gamma_4 = \begin{pmatrix} I_{\{2 \times 2\}} & 0 \\ 0 & -I_{\{2 \times 2\}} \end{pmatrix}, \quad \\
\quad \hookrightarrow \text{quad} \gamma_0 = I_{\{4 \times 4\}} = \begin{pmatrix} I_{\{2 \times 2\}} & 0 \\ 0 & I_{\{2 \times 2\}} \end{pmatrix}, \quad \text{quad } C = \\
\quad \hookrightarrow \gamma_2 \gamma_4 = \begin{pmatrix} -i \sigma_2 & 0 \\ 0 & i \sigma_2 \end{pmatrix}

```

Clearly we have $\gamma_\mu^\dagger = \gamma_\mu$ for $\mu = 1, 2, 3, 4, 5$ in the DeGrand-Rossi basis. We have γ_μ

- $\hookrightarrow$ anticommutation relations $\{\gamma_\mu, \gamma_\nu\} = 2\delta_{\mu\nu}$. Only γ_1 and γ_3
- $\hookrightarrow$ will generate a negative sign after the transpose. The auxiliary γ_0 is defined as the identity matrix
- $\hookrightarrow$, and the charge conjugation matrix $C = \gamma_2 \gamma_4$.

```

1 ---
2 name: pyquda-tool
3 description: >
4   PyQUDA tool usage skill. Generates Python code that calls PyQUDA to solve
5   quark propagators on lattice gauge configurations. Covers: configuration
6   loading, quark parameter setup
7   (Wilson/clover action, mass or kappa, clover coefficient, link smearing),
8   multigrid solver configuration, source construction (point, Gaussian
9   smearing with APE/HYP/stout links), propagator inversion, and residual
10  verification. Reads ensemble parameters from ensemble_registry.yaml.
11  Trigger on: "compute propagators", "solve propagator", "run inversions",
12  "call PyQUDA", "solve Dirac equation", or when
13  lqcd-physics-correlator has produced a propagator requirements list.
14  For pure-gauge observables (Wilson loops, Polyakov loops, etc.)
15  use the pyquda-gauge skill.
16 ---
17
18 # PyQUDA Tool Usage
19
20 ## Purpose
21
22 Translate a propagator specification into executable PyQUDA code
23 that reads gauge configurations and produces propagator or correlator data files.
24 This skill generates a **data production script only** — a single script that
25 computes and saves per-configuration results. It does not produce analysis code.

```

```

1 Calculate the two-point function of a pion with the operator  $\bar{u} \gamma_5 d$ .
2 Use the point source at position [0,0,0,0].
3 Invert all propagators using stout-smear gauge links with parameters (1, 0.125, 4).
4 Save the final result without any header or extra text to a txt file in the run directory.

```

```

1 workflow:
2   executor_static_check_rounds: 1
3   executor_dynamic_test_rounds: 1
4
5 script:
6   shell: /bin/bash
7
8 slurm:
9   partition: kshdclu16
10  nodes: 1
11  gres: dcu:4
12  exclude: f17r3n00
13  ntasks_per_node: 4

```

```

14 ntasks_per_socket: 1
15 exclusive: true
16 array: "10000-10850:50"
17
18 ensemble:
19   preset: C24P29
20   cfg_num: ["10000"]
21
22 runtime:
23   module_purge: true
24   modules:
25     - apps/git/2.30.2

```

```

γ
1 if __name__ == "__main__":
2     # Cases list
3     #
4     # Each entry: (label, sink_operator, source_operator)
5     #
6     # sink → first operator, sits at t (later time)
7     # source → second operator, sits at t=0 (earlier time) †
8     #
9     # The engine computes: sink(t) | source†(0)
10    # - For mesons: source† = Dirac adjoint  $\gamma(\cdot \Gamma^\dagger \cdot \gamma)$  applied automatically
11    # - For baryons: source† = Dirac adjoint  $\bar{0} = 0^\dagger \gamma \cdot$  applied automatically
12    #
13    #
14    #
15    # === Meson operator ===
16    #
17    # meson_operator(anti_flavor, quark_flavor, gamma):
18    #    $\bar{q}(q)(\text{anti\_flavor}) \cdot \Gamma(\text{gamma}) \cdot q(\text{quark\_flavor})$ 
19    #

```

```

1 from typing import Dict, Generator, List, Optional, Tuple, Union
2
3 from .operator import Operator
4 from .tensor import (
5     SpinColorTensorType,
6     QuarkFieldTensor,
7     SpinGammaTensor,
8     SpinProjectorTensor,
9     ColorDeltaTensor,
10    ColorEpsilonTensor,
11 )
12 from .adjacency import AdjacencyTerm
13 from .index import IndexMap
14
15
16 class WickTerm:
17     def __init__(
18         self,
19         factor: Union[int, float, complex],
20         tensors: List[Union[QuarkFieldTensor, SpinColorTensorType]],
21     ) -> None:
22         self.factor = factor

```



```

23     self.tensors: List[SpinColorTensorType] = []
24     self.quark_fields: List[QuarkFieldTensor] = []
25     quark_map: Dict[int, int] = {}
26     antiquark_map: Dict[int, int] = {}
27     self.index_map = IndexMap()
28     self.num_quark = 0
29     self.num_antiquark = 0
30     for tensor in reversed(tensors):

1  """two_pt.codegen — PyQUDA einsum format for 2-point functions.
2
3  Formats wick contraction output into PyQUDA contract() calls.
4  """
5
6  from pathlib import Path
7  try:
8      from ..wick_translate import _simplify_gammas
9      from .contract import ContractionTerm
10 except ImportError:
11     # Allow standalone operation (no parent package)
12     import sys
13     _p = str(Path(__file__).resolve().parent.parent)
14     if _p not in sys.path: sys.path.insert(0, _p)
15     from _wick_translate import _simplify_gammas
16     from contract import ContractionTerm
17
18
19 # Gamma matrix variable names recognized in PyQUDA code
20 _GAMMAS = frozenset(("G5", "g1", "g2", "g3", "g4", "gtg5", "Cg5", "I4", "Cg1"))
21
22
23 # =====
24 # PyQUDA einsum format
25 # =====
26
27 def pyquda_format_contract(t: ContractionTerm) -> str:
28     """Format ContractionTerm -> contract() string.
29     meson 2pt and baryon 2pt:
30         Meson 2pt: <O_snk . O_src^dag> = Tr[S_bwd . Gamma_snk . S_fwd . Gamma_bar_src]

```

公式 (1) (关联函数定义) 与 (2) (einsum 压缩编码) 为本报告核心物理公式, 来源与推导见第 2 节 C2/C6。

5 参考源清单

参考源	类型	内容/作用
prompts/planner/solve_system.md:1-14	仓库	C1: 物理学家 persona 与双模式
prompts/executor/generate_system.md:34-46	仓库	C1: 单文件骨架约束

参考源	类型	内容/作用
prompts/executor/generate_system.md:63-82	仓库	C1/C6: 工具强制与水印
prompts/planner/critique_system.md:6-9	仓库	C1: 批判标准
core_architecture/planner.py:72-112	仓库	C1/C4: solve/critique/rewrite
core_architecture/executor.py:102-159	仓库	C1/C4: generate_bundle
skills/lqcd-physics-correlator/SKILL.md:25-63	仓库	C2: 物理链与 γ 约定
skills/lqcd-physics-correlator/SKILL.md:88-98	仓库	C2: Wick/ γ 5-厄米/味对称
skills/lqcd-physics-spectrum/SKILL.md:1-20	仓库	C2: 谱分解分工
skills/pyquda-tool/SKILL.md:1-30	仓库	C2: PyQUDA 代码风格
skills/pyquda-gauge/SKILL.md:1-30	仓库	C2: 纯规范观测技能
utils/skill_utils.py:215-247	仓库	C2/C8: 技能注入装配
utils/skill_utils.py:249-262	仓库	C2/C8: reference 内联
utils/skill_utils.py:133-190	仓库	C8: SkillRouter 路由
configs/skills.yaml:1-10	仓库	C2/C5: 路由配置
benchmark/2pt_local/task/task_1.txt:1-5	仓库	C3: 2pt 任务原文
benchmark/3pt_baryon/task/task_1.txt:1-8	仓库	C3: 重子 3pt 任务
benchmark/wilson_loop/task/task_1.txt:1-6	仓库	C3: Wilson 圈任务
benchmark/2pt_local/standard/benchmark.py:1-30	仓库	C3/C6: 手写参考实现
README.md:108-118	仓库	C3: 70 任务分类
README.md:131-133	仓库	C3: Exact/sign-flip 判据
experiments/LQCD_Master_GPT_5.4/summary.md:8-30	仓库	C3/C9: 90.0% 结果
experiments/LQCD_Master_DeepSeek/summary.md:20-28	仓库	C3/C9: 80.0% 与失败表
core_architecture/orchestrator.py:74-96	仓库	C4: run() 三阶段
core_architecture/orchestrator.py:109-133	仓库	C4: checkpoint 循环
core_architecture/orchestrator.py:201-343	仓库	C4: 提交监视

参考源	类型	内容/作用
core_architecture/executor.py:165-197	仓库	C4: 缺字段重试
run.py:17-25	仓库	C5: CLI 参数
run.py:90-129	仓库	C5: main() 装配
configs/config.yaml:1-41	仓库	C5: Slurm/runtime 配置
core_architecture/executor.py:1113-1160	仓库	C5: 提交脚本渲染
utils/generate_einsum/two_pt/codegen.py:1-54	仓库	C6: einsum 格式化
utils/generate_einsum/wicklib/correlator.py:1-30	仓库	C6: Wick 引擎
utils/generate_einsum/hadron_operator.py:1-40	仓库	C6: 算符建模
utils/generate_einsum/two_pt/demo_2pt.py:22-118	仓库	C6: 驱动主流程（实测运行）
utils/static_check.py:10-25	仓库	C7: 静态分析入口
utils/static_check.py:186-265	仓库	C7: PyQUDA AST 分析
utils/llm_client.py:27-60	仓库	C10: LLM 客户端
utils/tool_client.py:33-99	仓库	C10: web/execute 工具
experiments/New_DA_diagonal/planner/plan.yaml:1	仓库	C4: 编排产物落盘实例

6 结论

结论

穷尽剖析结论：核心部分候选 13 个（深挖 6 / 浅析 4 / 排除 3，无未处理）。最强竞争力：① generate_einsum Wick 收缩引擎——把 Wick 定理自动化为正确 einsum 代码（实测 5 类粒子正确、自动处理 disconnected），是 2pt/3pt 高准确率的决定性依赖；② 双代理 + 三子阶段循环 + 技能路由注入——物理/ 代码双层评审 + 人机 checkpoint，支撑 70 任务基准 90.0%。二者构成”物理知识自动编译为正确代码”的完整链条，是本库区别于通用 agent 脚手架的核心竞争力。

参考背景

本库 docs/ 既有 pure 报告（pure_qcd_wick_codegen_20260815）独立剖析了 Wick 代码生成，与本报告 C6 实测互相印证：该引擎物理正确性与工程成熟度均可独立复现。

局限与风险

未验证：双代理真实 LLM 调用与 Slurm 提交未运行（只读分析，仅做语法检查 OK=69/FAIL=0 与 generate_einsum demo 实测）。**推断**：C12 评价/补充中的局限项（批判保守、sink 体积、无 schema 校验）均为推断。**排除项**：C11 生成大文件、C12 第三方报告、C13 通用脚手架——均非本库设计部分。**抽查**：experiments（5335 文件）与 10 万行级生成文件仅抽查/索引，未逐文件深读。